Laboratorio 2: Arquitectura del Computador

Michelle Sayas y Tulio Cordero

Agosto 2026

1. Introducción

El presente informe detalla la implementación de los algoritmos de ordenamiento **Quicksort** y **Merge Sort** en lenguaje ensamblador MIPS32. Se analiza el manejo de la pila, el uso de registros y la eficiencia de la arquitectura RISC en el entorno MARS.

2. Cuestionario

2.1. ¿Qué diferencias existen entre registros temporales ($t0–$t9) y registros guardados ($s0 $s7) y cómo se aplicó esta distinción en la práctica?

Diferencia: Los registros temporales no se preservan entre llamadas a funciones (la función que llama asume que pueden cambiar), mientras que los registros guardados sí deben preservarse (la función llamada debe guardarlos en la pila si planea modificarlos).

Aplicación: En tu mergeSort.asm, utilizaste $s0 para almacenar la variable mid. Como mergesort es recursiva, $s0 se guardó en la pila al inicio (sw $s0, 12($sp)) y se restauró al final. Los registros temporales (ej. $t0, $t1) se usaron libremente para iteradores y cálculos de direcciones sin guardarlos en la pila.

2.2. ¿Qué diferencias existen entre los registros $a0–$a3, $v0–$v1, $ra y cómo se aplicó esta distinción en la práctica?

Diferencia: $a0-$a3 son para pasar argumentos a las funciones. $v0-$v1 se usan para devolver valores de una función o indicar códigos de syscall. $ra almacena la dirección de retorno tras ejecutar una instrucción jal.

Aplicación: En quick_sort.asm, pasaste la dirección base del arreglo y los índices low y high mediante $a0, $a1 y $a2. Retornaste la posición del pivote desde la función partition utilizando $v0. El registro $ra fue vital guardarlo en la pila en las funciones recursivas para no perder el rastro de la ejecución al llamar a la misma función nuevamente.

2.3. Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

El uso de registros mejora drásticamente el rendimiento ya que el acceso a la RAM toma múltiples ciclos de reloj. Al mantener punteros e índices en registros lógicos, se reduce la latencia de ejecución.

En ambos algoritmos, al mantener variables iteradoras (como i, j, k) y direcciones precalculadas en registros lógicos, se minimiza el uso de costosas instrucciones lw y sw, lo que acelera exponencialmente el tiempo total de procesamiento.

2.4. ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Las instrucciones de bucles y saltos condicionales (bge, bgt, j) alteran el flujo secuencial, lo que en la arquitectura segmentada (pipeline) de MIPS32 puede generar riesgos de control (control hazards).

Si el procesador falla en predecir un salto, debe vaciar las instrucciones precargadas, lo que introduce ciclos de parada (stalls) y reduce la eficiencia en bucles anidados intensivos.

2.5. Complejidad Computacional

- **Quicksort**: Promedio $O(n \log n)$, peor caso $O(n^2)$.
- **Merge Sort**: Garantiza $O(n \log n)$ en todos los casos.
- **Implicación en MIPS**: Merge Sort requiere más gestión de memoria en la pila para arreglos temporales, mientras que Quicksort es más eficiente en uso de memoria al ser *in-place*.

2.6. ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

Consta de 5 fases:
1. **IF**: Búsqueda de instrucción.

2. **ID**: Decodificación y lectura de registros.

3. **EX**: Ejecución en la ALU.

4. **MEM**: Acceso a memoria de datos.

5. **WB**: Escritura de resultados en registros.

2.7. ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

Predominaron las instrucciones tipo I (ej. lw, sw, addi, bge).

Los algoritmos de ordenamiento se basan en comparar datos (saltos condicionales iterativos) y mover datos en arreglos (lectura/escritura en memoria base + offset inmediato), lo cual es el propósito principal de las instrucciones I.

2.8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (j, beq, bne) en lugar de usar estructuras lineales?

Un abuso de saltos (j, beq, bne) fragmenta la ejecución. Esto incrementa los fallos de predicción de saltos en el pipeline del procesador, obligando a descartar el trabajo ya iniciado y reduciendo el rendimiento general frente a un código lineal.

2.9. Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

Su arquitectura Load/Store restringe las operaciones de memoria solo a lw y sw. Toda la aritmética se hace entre registros. Esto permite que el procesador ejecute instrucciones de manera muy uniforme y rápida (1 ciclo por instrucción en el mejor de los casos), ideal para las rápidas comparaciones e intercambios numéricos en ordenamientos.

2.10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

- **Recuadro Azul (Text Segment):** Es el área donde se muestra el código ensamblado. Aquí puedes ver la dirección de memoria de cada instrucción, el código de máquina (hexadecimal), la instrucción básica y la línea de código fuente original. Es fundamental para seguir el flujo del programa.
- **Recuadro Amarillo (Data Segment):** Muestra el contenido de la memoria de datos (RAM). Permite visualizar cómo se almacenan las variables, arreglos y cadenas de texto en direcciones específicas (como 0x10010000). En la imagen se ven valores hexadecimales que representan los datos del programa.
- **Recuadro Rojo (Registers):** Esta tabla muestra el estado actual de todos los registros de la CPU MIPS (desde $zero hasta $ra). Indica el nombre del registro, su número y el valor que contiene en tiempo real. Es vital para verificar cálculos intermedios.
- **Recuadro Verde (Botón Step/Ejecutar):** Encierra el botón de "Step"(Paso a paso). Sirve para ejecutar una sola instrucción a la vez, permitiendo al programador observar el efecto exacto de cada línea sobre los registros y la memoria.

- **Recuadro Rosa (Mars Messages / Run I/O):** Es la consola de salida y mensajes. Aquíaparecen errores de ensamblado y, en la pestaña Run I/O", se muestra la interacción con el usuario (impresión de texto o lectura de datos por teclado).

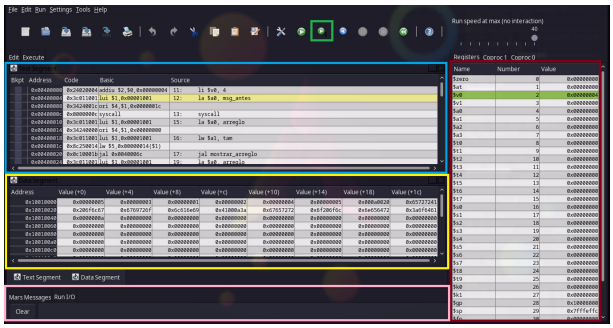


Figura 1: Enter Caption

2.11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

El panel de Registros (Registers) en combinación con la vista del Segmento de Datos (Data Segment). Ver en tiempo real la memoria RAM reflejó cómo los números iban cambiando de posición física, revelando instantáneamente si las lógicas de intercambio (partition o merge) estaban funcionando.

.

2.12. Visualización del camino de datos para una instrucción tipo I (lw)

Para visualizar el camino de datos (Data Path), se debe abrir la herramienta MIPS X-Ray desde el menú Tools. Una vez conectada al programa mediante el botón Çonnect to MIPS", al ejecutar una instrucción tipo R como add, se observa en el diagrama:

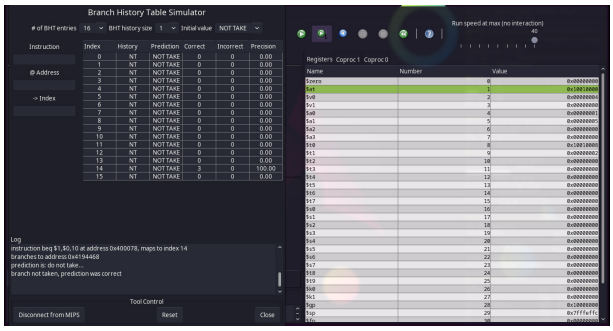


Figura 2: Enter Caption

- **Flujo de Registros:** Se iluminan las líneas que van desde el bloque REGISTERS hacia la ALU.
- **Operación en ALU:** El bloque central (ALU) muestra el procesamiento de los datos.

- **Retroalimentación (Write Back):** Se visualiza una línea que regresa desde la salida de la ALU hacia la entrada de "Write Data.[en] los registros, indicando que el resultado se guarda en el registro destino (rd).
- **Exclusión de Memoria:** La línea que pasa por DATA MEMORY se muestra inactiva o el multiplexor final selecciona la salida de la ALU, ya que estas instrucciones no acceden a la memoria RAM.

2.13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

Utilizando la misma herramienta MIPS X-Ray, la visualización para una instrucción de carga como lw (load word) cambia significativamente respecto a las de tipo R:

- **Cálculo de Dirección:** Se observa cómo el valor de un registro base y el valor inmediato (que pasa por el bloque SIGN EXTEND) entran a la ALU para calcular la dirección de memoria.
- **Acceso a Memoria:** A diferencia de las tipo R, aquí se ilumina activamente el bloque DATA MEMORY. El camino de datos muestra el flujo entrando a la memoria para leer un dato.
- **Escritura en Registro:**El camino de datos continúa desde la salida de la DATA MEMORY, pasa por el multiplexor final y llega al bloque REGISTERS para guardar el valor en el registro destino (rt).

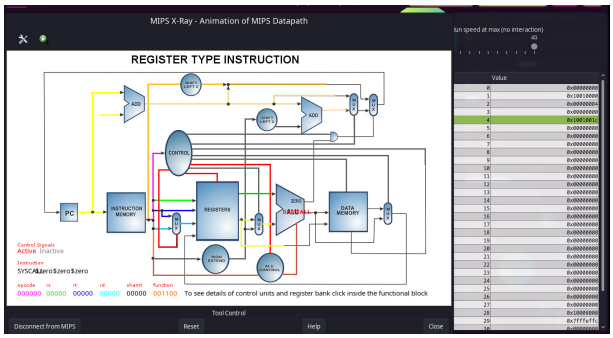


Figura 3: Enter Caption

3. Justificación del Algoritmo Alternativo

Se eligió Merge Sort porque representa un contraste excelente frente al Quicksort. Ambos utilizan la estrategia "Divide y Vencerás", pero Merge Sort ilustra un reto mayor en ensamblador: la reserva y liberación dinámica de memoria en la pila para arreglos temporales durante la fase de mezcla.

4. Análisis y Discusión de los Resultados

La arquitectura RISC de MIPS simplifica la ejecución mediante un conjunto de instrucciones reducido, pero traslada la responsabilidad de la gestión de memoria y registros

al programador. La correcta implementación de la recursividad fue la clave para el éxito de esta práctica.

Ambos algoritmos se ejecutaron de manera íntegra, logrando el objetivo y confirmando que un manejo diligente de punteros de memoria y control de flujo da como resultado programas altamente eficientes a nivel de hardware.